

KONEČNÉ AUTOMATY

V této kapitole se budeme věnovat základům *teorie automatů*. Konečné automaty jsou jednoduché abstraktní stroje, které zpracovávají vstupní slovo znak po znaku a na konci rozhodnou, zda jej přijmou, nebo odmítnou. Množiny přijímaných slov tvoří tzv. *formální jazyky*.

Nejdříve si zavedeme potřebné operace nad slovy a jazyky. Poté se podíváme na deterministické a nedeterministické konečné automaty, regulární výrazy a formální gramatiky. Ukáže se, že konečné automaty, regulární výrazy a regulární gramatiky jsou tři různé způsoby popisu stejné třídy jazyků. Později tuto teorii využijeme při lexikální analýze v kapitole o kompilátorech.

1 Úvod

Mnoho programů lze popsat pomocí konečného počtu *stavů* a pravidel, podle kterých se mezi nimi přechází. Obvyčejný spínač má například dva stavy: zapnuto a vypnuto. Každé stisknutí jej přesune do druhého stavu.

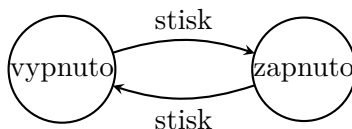


Figure 1: Jednoduchý stavový model spínače.

Podobně můžeme po jednotlivých znacích rozpoznávat klíčové slovo **then**. Stav si pamatuje, jak velkou část slova jsme již přečetli.

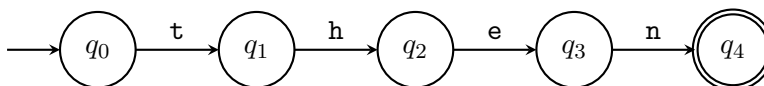


Figure 2: Schéma automatu rozpoznávajícího slovo **then**.

Konečný automat tedy dostane na vstupu konečnou posloupnost znaků, které říkáme *slovo*. Slovo čte zleva doprava a po každém znaku může změnit svůj stav. Po přečtení celého vstupu vydá odpověď: slovo přijme, nebo odmítne.

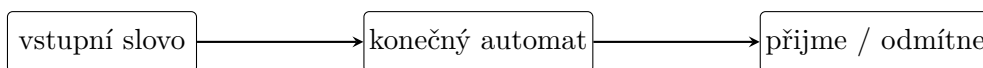


Figure 3: Zjednodušené schéma konečného automatu.

Automat slova *rozpoznává*: rozhoduje, která z nich patří do popisovaného jazyka. Později si ukážeme gramatiky, které naopak slova *generují*.

2 Formální jazyky

Než se pustíme do přesné definice automatu, potřebujeme si ujasnit, co vlastně automat dostává na vstupu a jak budeme popisovat množiny přípustných vstupů.

Definice 2.1 (Abeceda, slovo a jazyk). Mějme konečnou neprázdnou množinu znaků Σ , kterou nazýváme *abecedou*. Pak

- *slovo (řetězec)* je libovolná konečná posloupnost znaků z Σ ;
- *prázdné slovo* je slovo neobsahující žádný znak, značíme je λ ;
- množinu všech slov nad Σ značíme Σ^* ;
- množinu všech neprázdných slov značíme $\Sigma^+ = \Sigma^* \setminus \{\lambda\}$;
- *jazyk nad abecedou* Σ je libovolná množina $L \subseteq \Sigma^*$;
- délku slova w značíme $|w|$ a počet výskytů znaku a ve slově w značíme $|w|_a$.

Symbol λ není znakem abecedy. Je to označení posloupnosti délky nula. Proto pro každé slovo w platí $\lambda w = w\lambda = w$.

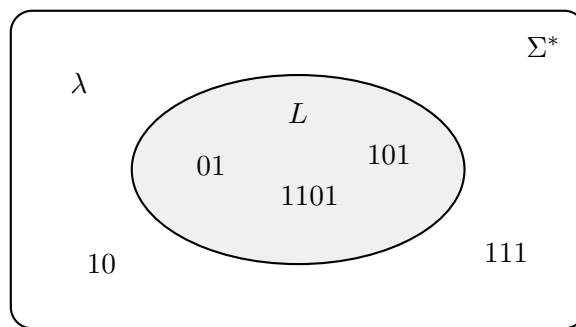


Figure 4: Jazyk L je libovolná podmnožina množiny všech slov Σ^* .

Definice 2.2 (Operace nad slovy). Necht $u = u_1u_2 \dots u_n$ a $v = v_1v_2 \dots v_m$ jsou slova nad Σ .

- Jejich *konkatenace (zřetězení)* je slovo

$$uv = u_1u_2 \dots u_nv_1v_2 \dots v_m.$$

- Mocniny slova definujeme vztahy $u^0 = \lambda$ a $u^{k+1} = u^k u$ pro $k \in \mathbb{N}_0$.

Definice 2.3 (Operace nad jazyky). Pro jazyky $K, L \subseteq \Sigma^*$ definujeme

$$\begin{aligned} K \cup L &= \{w \in \Sigma^* : w \in K \text{ nebo } w \in L\}, \\ KL &= \{uv : u \in K, v \in L\}, \\ L^0 &= \{\lambda\}, \quad L^{n+1} = L^n L, \\ L^* &= \bigcup_{n=0}^{\infty} L^n. \end{aligned}$$

Množině L^* říkáme *Kleeneho uzávěr* jazyka L .

Příklad 2.4. Mějme abecedu $\Sigma = \{a, b\}$ a slova $u = aabb$, $v = bba$.

- $|u| = 4$, $|u|_a = 2$ a $uv = aabbbba$.

- $v^3 = bbabbabba$.
- $\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$.
- Jazyk všech slov délky nejvýše dva je

$$\{w \in \Sigma^* : |w| \leq 2\} = \{\lambda, a, b, aa, ab, ba, bb\}.$$

Jazyky mohou být zadány výčtem, vlastností nebo jiným popisem. Uvedme si tři o něco zajímavější příklady nad binární abecedou $\Sigma = \{0, 1\}$:

$$\begin{aligned} L_{\text{suff}} &= \{w \in \Sigma^* : w \text{ končí na } 01\} = \{u01 : u \in \Sigma^*\}, \\ L_{\text{bez } 11} &= \{w \in \Sigma^* : w \text{ neobsahuje podslovo } 11\}, \\ L_{\text{eq}} &= \{w \in \Sigma^* : |w|_0 = |w|_1\}. \end{aligned}$$

První dva jazyky budou regulární a později pro ně sestrojíme automaty. Třetí jazyk regulární není: konečný automat si neumí pamatovat neomezeně velký rozdíl mezi počtem nul a jedniček.

2.1 Rozhodovací problémy jako jazyky

V minulé kapitole jsme vstupy rozhodovacích problémů kódovali pomocí nul a jedniček. Rozhodovací problém tak můžeme ztotožnit s jazykem všech zakódovaných vstupů, pro které je odpověď kladná. Například

$$\text{SAT} = \{\langle \varphi \rangle : \varphi \text{ je splnitelná formule v CNF}\},$$

kde $\langle \varphi \rangle \in \{0, 1\}^*$ značí binární kód formule φ . Tento pohled nám později umožňuje studovat výpočetní problémy pomocí strojů, které přijímají jazyky.

Jazyk obsahuje pouze kladné instance. Řetězec kódující nesplnitelnou formuli do jazyka SAT nepatří; stejně tak do něj nezařazujeme řetězce, které vůbec nejsou platným kódem formule. Algoritmus rozhodující jazyk musí pro každý vstup skončit a správně určit, zda do jazyka patří.

Konkrétní způsob kódování může změnit délku vstupu o konstantní nebo polynomiální faktor, nemá však měnit samotnou odpověď problému. Díky tomu můžeme oddělit matematický objekt od jeho zápisu a porovnávat výpočetní modely jednotným jazykovým pohledem: automat či stroj jazyk buď přijímá, nebo odmítá.

3 Deterministické konečné automaty

Definice 3.1 (Deterministický konečný automat). *Deterministický konečný automat* (zkráceně DFA) je uspořádaná pětice

$$A = (Q, \Sigma, \delta, q_0, F),$$

kde

- Q je konečná množina stavů,
- Σ je vstupní abeceda,
- $\delta : Q \times \Sigma \rightarrow Q$ je přechodová funkce, která každé dvojici stavu a znaku přiřazuje právě jeden následující stav,
- $q_0 \in Q$ je počáteční stav a

- $F \subseteq Q$ je množina přijímajících stavů.

Při kreslení někdy kvůli přehlednosti vynecháme *odpadní stav*: všechny nezakreslené přechody si pak představujeme vedené do tohoto nepřijímajícího stavu, který má smyčku pro každý znak. Formálně je tak přechodová funkce stále definována pro každou dvojici.

Automat zakreslujeme jako orientovaný graf. Stavy jsou vrcholy, přechody hrany s popisky. Počáteční stav označíme šipkou vedoucí „odnikud“ a přijímající stavy dvojitým okrajem.

Výpočet automatu. Na začátku se automat nachází ve stavu q_0 . Vstupní slovo čte zleva doprava. Nachází-li se ve stavu q a čte znak a , přejde do stavu $\delta(q, a)$. Po přečtení celého slova přijme právě tehdy, když skončil ve stavu $z \in F$.

Definice 3.2 (Jazyk automatu). Jazykem automatu A nad abecedou Σ rozumíme množinu

$$L(A) = \{w \in \Sigma^* : A \text{ přijme slovo } w\}.$$

Příklad 3.3 (Slova začínající na 01). Automat na obrázku 5 nejprve vyžaduje znaky 0 a 1. Jakmile je přečte, zůstane v přijímajícím stavu bez ohledu na zbytek vstupu. Jeho jazyk je

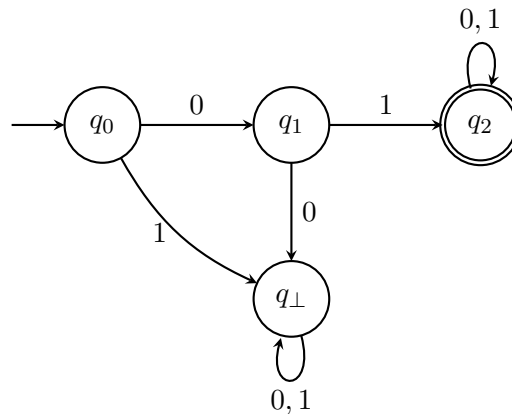


Figure 5: DFA přijímající právě slova začínající na 01.

$$L(A) = \{01u : u \in \{0, 1\}^*\}.$$

Slovo 0110 přijme, zatímco 001 odmítne už při čtení druhého znaku.

Složitější příklady.

Příklad 3.4 (Slova končící na 01). Nyní už nestačí zkontrolovat první dva znaky. Automat si musí průběžně pamatovat nejdelší konec dosud přečtené části, který by mohl být začátkem slova 01:

- q_0 : poslední znak není 0 nebo jsme zatím nic nepřečetli;
- q_1 : poslední znak je 0;
- q_2 : poslední dva znaky jsou 01.

Příklad 3.5 (Sudý počet nul a lichý počet jedniček). Potřebujeme si pamatovat dvě informace, každou se dvěma možnostmi. Dostaneme proto čtyři stavy $q_{SS}, q_{SL}, q_{LS}, q_{LL}$; první index udává paritu počtu nul a druhý paritu počtu jedniček. Znak 0 změní první index, znak 1 druhý. Jediným přijímajícím stavem je q_{SL} .

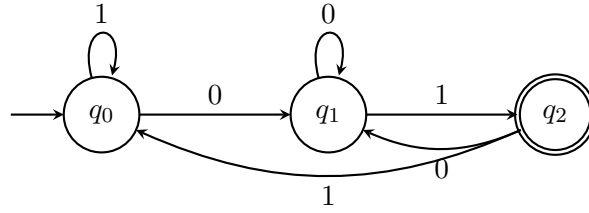


Figure 6: DFA pro jazyk $L_{\text{suff}} = \{u01 : u \in \{0,1\}^*\}$.

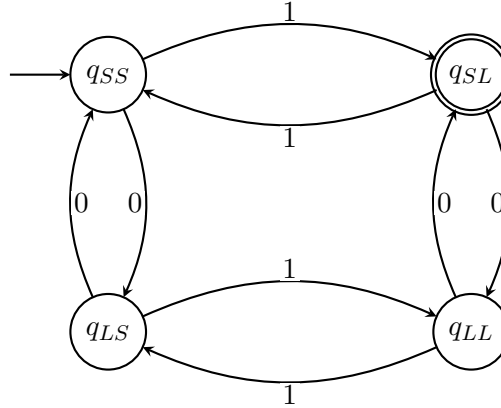


Figure 7: DFA přijímající slova se sudým počtem nul a lichým počtem jedniček.

Příklad 3.6 (Binární čísla dělitelná třemi). Čteme-li binární číslo zleva doprava a dosud přečtená část dává po dělení třemi zbytek r , pak po připsání bitu $b \in \{0,1\}$ dostaneme zbytek čísla $2r + b$. Stačí si tedy pamatovat zbytek 0, 1 nebo 2. Kvůli vyloučení prázdného slova používáme zvláštní počáteční stav

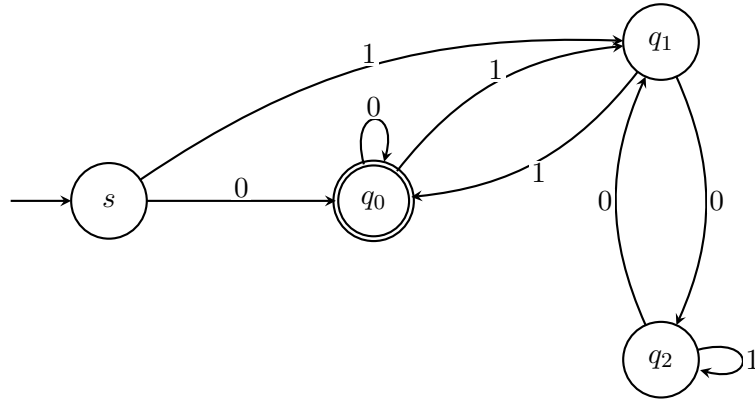


Figure 8: DFA pro neprázdné binární zápisy čísel dělitelných třemi.

s. Automat například přijme $110_2 = 6$, ale odmítne $101_2 = 5$.

4 Nedeterministické konečné automaty

U deterministického automatu vede pro daný stav a znak nejvýše jeden přechod. Někdy je však návrh mnohem jednodušší, když automatu dovolíme několik možností a představíme si, že vyzkouší všechny paralelně.

Poznámka 4.1. Deterministické a nedeterministické konečné automaty systematicky popsali Michael O. Rabin a Dana Scott. Za článek z roku 1959 a navazující práci o konečných automatech obdrželi

v roce 1976 Turingovu cenu. Jejich výsledek ukázal, že nedeterminismus zápis zkracuje, ale u konečných automatů nezvětšuje výpočetní sílu.

Definice 4.2 (Potenční množina). Nechť M je množina. *Potenční množinou* množiny M nazveme množinu všech jejích podmnožin. Značíme ji

$$\mathcal{P}(M) = \{N : N \subseteq M\}.$$

Například pro $M = \{q_0, q_1\}$ dostaneme

$$\mathcal{P}(M) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}.$$

Definice 4.3 (Nedeterministický konečný automat). *Nedeterministický konečný automat* (zkráceně NFA) je pětice

$$A = (Q, \Sigma, \delta, q_0, F),$$

kde Q , Σ , q_0 a F mají stejný význam jako u DFA a

$$\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$$

přiřazuje dvojici stavu a znaku množinu možných následujících stavů.

Přečte-li NFA ve stavu q znak a , výpočet se rozvětví do všech stavů z $\delta(q, a)$. Je-li tato množina prázdná, daná větev zanikne. Automat přijme vstupní slovo právě tehdy, když *alespoň jedna* větev po přečtení celého slova skončí v přijímajícím stavu.

Příklad 4.4 (Slova obsahující podslovo 101). Počáteční stav q_0 zpracovává libovolný začátek slova. Při každé jedničce může automat zároveň „uhádnout“, že právě začalo hledané podslovo, a spustit novou větev ve stavu q_1 . Pro slovo 01101 vznikne přijímající větev, která po posledních třech znacích projde

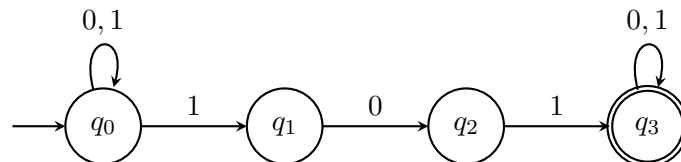


Figure 9: NFA přijímající právě slova obsahující podslovo 101.

stavy q_1, q_2, q_3 .

Příklad 4.5 (Třetí znak od konce je 1). NFA může při každé přečtené jedničce odhadnout, že do konce zbývají právě dva znaky. Pak už pouze odpočítá dva další přechody. Tento automat má jen čtyři stavy.

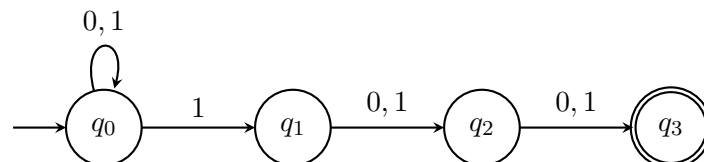


Figure 10: NFA pro jazyk slov, jejichž třetí znak od konce je 1.

Ekvivalentní DFA si musí pamatovat všechny možné konce délky tři a potřebuje osm stavů.

Příklad 4.6 (Jednoduchý zápis desetinného čísla). Automat na obrázku 11 přijímá neprázdný blok číslic a případně desetinnou tečku následovanou dalším neprázdným blokem číslic. Přijme tedy 17 a 17.25, ale odmítne .25, 17. i 1.2.3.

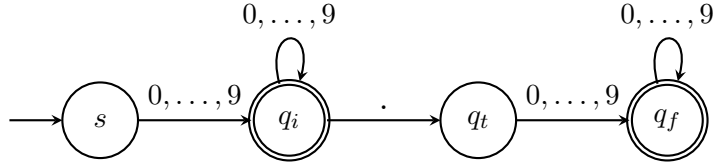


Figure 11: Konečný automat pro jednoduchý zápis nezáporného desetinného čísla.

4.1 Vztah DFA a NFA

Nedeterminismus sice může výrazně zjednodušit zápis automatu, ale nerozšíří množinu jazyků, které konečné automaty dokážou přijímat.

Věta 4.7 (Podmnožinová konstrukce). Ke každému NFA A existuje DFA B takový, že $L(A) = L(B)$.

Proof. Mějme NFA $A = (Q, \Sigma, \delta, q_0, F)$. Jeden stav nového DFA bude popisovat celou množinu stavů, v nichž se NFA může po přečtení stejného začátku slova nacházet. Položíme tedy

$$B = (\mathcal{P}(Q), \Sigma, \delta', \{q_0\}, F'),$$

kde

$$\delta'(S, a) = \bigcup_{q \in S} \delta(q, a)$$

a množina $S \subseteq Q$ je přijímajícím stavem právě tehdy, když $S \cap F \neq \emptyset$.

Indukcí podle délky přečteného slova dostaneme, že stav DFA B je vždy přesně množinou všech stavů, ve kterých mohou být jednotlivé větve NFA A . DFA proto přijme právě tehdy, když alespoň jedna větev NFA skončí v F . $\square$

Každý DFA je zároveň NFA, jen všechny množiny $\delta(q, a)$ mají nejvýše jeden prvek. Oba modely tedy přijímají stejné jazyky. Cena za odstranění nedeterminismu může být vysoká: NFA s n stavy může podmnožinovou konstrukcí vytvořit DFA až s 2^n stavy.

4.2 Přechody bez čtení znaku

Pro důkaz Kleeneho věty se nám bude hodit ještě drobné technické rozšíření.

Definice 4.8 (λ -NFA). λ -NFA je NFA s přechodovou funkcí

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q),$$

který smí navíc použít přechod označený λ . Při takovém přechodu změní stav, aniž by přečetl znak vstupu.

Takový automat stále nepřijímá nic navíc. Pro množinu stavů S označme $E(S)$ množinu všech stavů dosažitelných ze S pouze pomocí nulového nebo více λ -přechodů. Obyčejný NFA sestojíme tak, že pro každý stav q a znak a položíme

$$\delta'(q, a) = E\left(\bigcup_{r \in E(\{q\})} \delta(r, a)\right)$$

a stav q prohlásíme za přijímající, pokud $E(\{q\}) \cap F \neq \emptyset$. Nový automat před každým znakem i po něm provede předem všechny možné λ -přechody, takže přijímá tentýž jazyk.

4.3 Regulární jazyky

Definice 4.9 (Regulární jazyk). Jazyk L nazveme *regulární*, pokud existuje DFA A takový, že $L = L(A)$.

Podle věty 4.7 můžeme v definici stejně dobře použít NFA nebo λ -NFA. Ne každý jazyk je však regulární.

Věta 4.10. Jazyk

$$L_{\text{eq}} = \{w \in \{0, 1\}^* : |w|_0 = |w|_1\}$$

není regulární.

Proof. Předpokládejme pro spor, že existuje DFA A s m stavy a $L(A) = L_{\text{eq}}$. Uvažujme $m + 1$ slov

$$\lambda, 0, 00, \dots, 0^m.$$

Automat má jen m stavů, takže po přečtení dvou různých slov 0^i a 0^j , kde $0 \leq i < j \leq m$, musí být ve stejném stavu.

Nyní k oběma slovům připojme stejný konec 1^i . Automat z téhož stavu čte tentýž zbytek, a proto buď přijme obě slova, nebo obě odmítne. Jenže

$$0^i 1^i \in L_{\text{eq}}, \quad 0^j 1^i \notin L_{\text{eq}}.$$

Automat by je musel rozlišit. Dostáváme spor. □

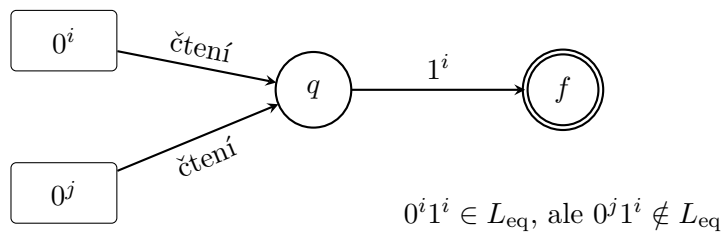


Figure 12: Po slovech 0^i a 0^j je automat ve stejném stavu, takže stejný konec 1^i už nedokáže rozlišit.

5 Regulární výrazy

Automat je názorný, ale pro větší jazyky se rychle rozroste. Regulární výraz popisuje tentýž druh jazyků stručněji: skládá malé jazyky pomocí sjednocení, zřetězení a Kleeneho hvězdy.

Definice 5.1 (Regulární výraz). Mějme abecedu Σ . Množinu regulárních výrazů nad Σ , značenou $\text{RegE}(\Sigma)$, definujeme následovně:

1. $\emptyset$, λ a každý znak $a \in \Sigma$ jsou regulární výrazy;
2. jsou-li $R, S \in \text{RegE}(\Sigma)$, pak také

$$(R \mid S), \quad (RS), \quad (R^*)$$

jsou regulární výrazy;

3. nic dalšího regulárním výrazem není.

Každému regulárnímu výrazu přiřadíme jazyk:

$$\begin{aligned} L(\emptyset) &= \emptyset, & L(\lambda) &= \{\lambda\}, & L(a) &= \{a\}, \\ L(R \mid S) &= L(R) \cup L(S), & L(RS) &= L(R)L(S), & L(R^*) &= L(R)^*. \end{aligned}$$

Řekneme, že výraz R popisuje jazyk $L(R)$.

Závorky se obvykle vynechávají. Nejvyšší prioritu má hvězda, potom zřetězení a nakonec sjednocení. Výraz $01^* \mid 10$ tedy znamená $(0(1^*)) \mid (10)$.

Příklad 5.2. Nad abecedou $\Sigma = \{0, 1\}$ platí:

1. $(0 \mid 1)^*$ popisuje všechna binární slova;
2. $(0 \mid 1)^*01$ popisuje slova končící na 01;
3. $0^*10^*10^*$ popisuje slova s právě dvěma jedničkami;
4. $0^*(10^*10^*)^*$ popisuje slova se sudým počtem jedniček;
5. $(0 \mid 10)^*(\lambda \mid 1)$ popisuje slova bez dvou jedniček vedle sebe;
- 6.

$$0(0 \mid 1)^*0 \mid 1(0 \mid 1)^*1 \mid 0 \mid 1$$

popisuje neprázdná slova, která začínají a končí stejným znakem.

U složitějšího regulárního výrazu se vyplatí nejdřív slovně říct, co znamenají jeho části. Například v $0^*(10^*10^*)^*$ přidá každý průchod vnější hvězdou právě dvě jedničky; nuly mohou být kdekoli mezi nimi.

5.1 Kleeného věta

Regulární výrazy nejsou jen pohodlnou zkratkou pro několik oblíbených jazyků. Popisují přesně totéž co konečné automaty.

Poznámka 5.3. Stephen Cole Kleene zavedl ve 50. letech 20. století algebraický popis množin přijímaných konečnými automaty. Jeho hvězda R^* vyjadřuje libovolný konečný počet opakování a dodnes se používá v regulárních výrazech programovacích jazyků i textových nástrojů.

Věta 5.4 (Kleeného věta). Jazyk je regulární právě tehdy, když jej popisuje nějaký regulární výraz.

Proof. Dokážeme obě implikace konstrukcí.

Z regulárního výrazu na automat. Postupujeme indukcí podle stavby výrazu. Pro základní výrazy $\emptyset$, λ a $a \in \Sigma$ sestrojíme λ -NFA na obrázku 13. Automat pro $\emptyset$ nemá mezi počátečním a přijímajícím stavem žádnou cestu, automat pro λ má jediný λ -přechod a automat pro a jediný přechod označený a .

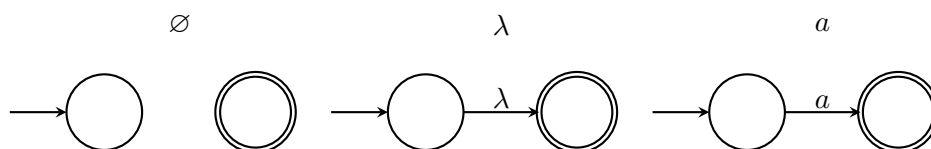


Figure 13: Základní automaty pro výrazy $\emptyset$, λ a a .

Předpokládejme nyní, že už máme automaty pro výrazy R a S . Jejich stavy případně přejmenujeme, aby se množiny stavů nepřekrývaly. Navíc můžeme bez újmy předpokládat, že každý z nich má právě jeden přijímající stav bez odchozích přechodů: stačí přidat nový přijímající stav a ze všech původních přijímajících stavů do něj vést λ -přechod.

Pro sjednocení $R \mid S$ přidáme nový počáteční a nový přijímající stav. Z nového začátku se automat λ -přechodem vydá do automatu pro R nebo S a z jejich konců přejde do nového přijímajícího stavu.

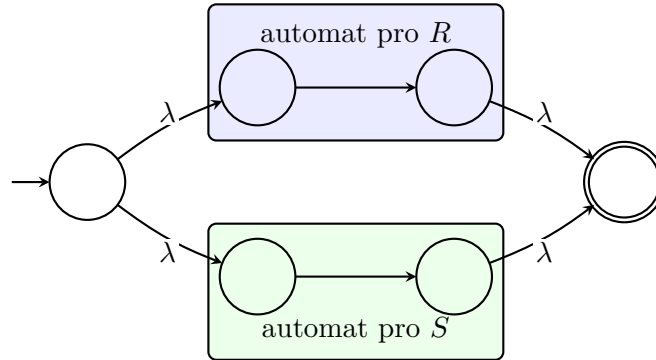


Figure 14: Konstrukce automatu pro sjednocení $R \mid S$.

Pro zřetězení RS spojíme přijímající stav automatu pro R λ -přechodem s počátečním stavem automatu pro S . První část vstupu tedy musí přijmout automat pro R a zbytek automat pro S .

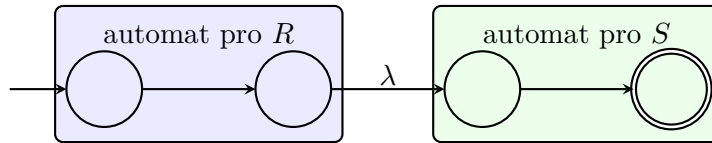


Figure 15: Konstrukce automatu pro zřetězení RS .

Pro R^* přidáme nový začátek a konec. Přímý λ -přechod mezi nimi přijme nula opakování. Další přechody umožní spustit automat pro R , po jednom průchodu skončit, nebo se vrátit a spustit jej znovu.

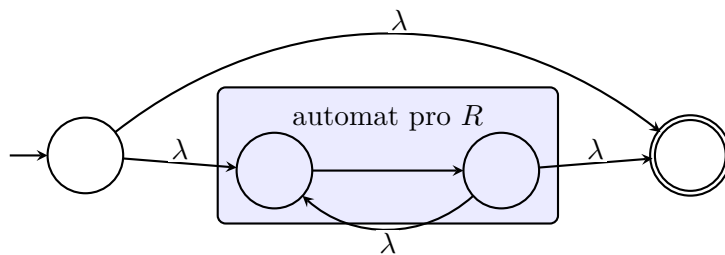


Figure 16: Konstrukce automatu pro Kleeneho hvězdu R^* .

Každá konstrukce přijímá právě jazyk odpovídající operace. Indukce tedy pro každý regulární výraz vytvoří ekvivalentní λ -NFA. Přechody bez čtení znaku umíme odstranit a podmnožinovou konstrukcí dostaneme DFA. Jazyk výrazu je proto regulární.

Z automatu na regulární výraz. Mějme DFA $A = (Q, \Sigma, \delta, q_s, F)$ a očísľujme jeho stavy

$$Q = \{q_1, \dots, q_n\}.$$

Pro dvojici stavů q_i, q_j a číslo $k \in \{0, \dots, n\}$ sestojíme regulární výraz $R_{ij}^{(k)}$. Bude popisovat právě slova, po jejichž přečtení se automat dostane z q_i do q_j a všechny mezilehlé stavy cesty patří do množiny $\{q_1, \dots, q_k\}$. Počáteční ani koncový stav se za mezilehlý nepovažují.

Pro $k = 0$ nesmíme použít žádný mezilehlý stav. Výraz $R_{ij}^{(0)}$ proto vytvoříme jako sjednocení všech znaků a , pro které $\delta(q_i, a) = q_j$. Je-li $i = j$, přidáme do sjednocení také λ ; není-li co sjednotit, použijeme $\emptyset$.

Pro $k \geq 1$ položíme

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \mid R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^*R_{kj}^{(k-1)}. \quad (1)$$

Každá cesta povolená pro $R_{ij}^{(k)}$ buď stav q_k jako mezilehlý vůbec nepoužije, a pak ji popisuje první člen, nebo jej použije. Ve druhém případě cestu rozřízneme při prvním příchodu do q_k a při posledním odchodu z něj. Dostaneme cestu z q_i do q_k , libovolný počet návratů z q_k do něj samého a cestu z q_k do q_j ; žádná z těchto částí už nemá q_k uvnitř. Právě to popisuje druhý člen vztahu (1). Obráceně lze části popsané kterýmkoliv členem vždy spojit v povolenou cestu. Rekurence je tedy správná.

Po dosazení $k = n$ dovolíme jako mezilehlé všechny stavy automatu. Hledaný regulární výraz je sjednocení

$$\bigcup_{q_j \in F} R_{sj}^{(n)},$$

kde q_s je počáteční stav. Je-li $F = \emptyset$, vezmeme místo sjednocení výraz $\emptyset$. Jeho jazyk tvoří právě slova, která dovedou automat z počátečního stavu do některého přijímajícího stavu. Je tedy roven $L(A)$. $\square$

Obě poloviny důkazu jsou zároveň algoritmy. Z výrazu umíme mechanicky postavit automat a z automatu zase výraz. Výsledek může být mnohem větší než zadání; věta slibuje existenci a postup, nikoliv vždy hezkému výsledku.

Regulární výrazy v praxi. Programovací jazyky a nástroje pro hledání v textu často používají rozšířené regulární výrazy. Zápisy jako $[0-9]$ nebo $+$ bývají jen zkratkami pro naše tři operace. Některé nástroje však přidávají třeba zpětné odkazy na dříve nalezenou část textu. Takový „regex“ už může popisovat i neregulární jazyk, a není tedy regulárním výrazem ve smyslu této kapitoly.

6 Gramatiky

Automat dostane hotové slovo a rozhodne, zda do jazyka patří. Gramatika jde opačným směrem: z počátečního symbolu slova jazyka postupně vyrábí.

Poznámka 6.1. Noam Chomsky v 50. letech 20. století uspořádal formální gramatiky do hierarchie podle povoleného tvaru pravidel. Původní motivací byl popis přirozených jazyků, stejné pojmy se však staly základem syntaxe programovacích jazyků a konstrukce překladačů.

Definice 6.2 (Gramatika). *Gramatika* je čtveřice

$$G = (V, T, P, S),$$

kde

- V je konečná množina *neterminálů*,
- T je konečná množina *terminálů*, přičemž $V \cap T = \emptyset$,

- P je konečná množina *přepisovacích pravidel*. Pravidlo zapisujeme ve tvaru $\alpha \rightarrow \beta$: na levé straně stojí řetězec terminálů a neterminálů obsahující alespoň jeden neterminál, na pravé straně může stát libovolný řetězec terminálů a neterminálů, případně prázdné slovo λ ,
- $S \in V$ je *počáteční neterminál*.

Levá strana pravidla tedy musí obsahovat alespoň jeden neterminál, pravá strana může být i prázdné slovo λ . Terminály už dále nepřepisujeme; budou tvořit výsledné slovo.

Definice 6.3 (Odvození a jazyk gramatiky). Řekneme, že se slovo $u\alpha v$ *přímo odvodí* na $u\beta v$, a píšeme

$$u\alpha v \Rightarrow_G u\beta v,$$

jestliže $\alpha \rightarrow \beta$ je pravidlo gramatiky. Zápis

$$x \Rightarrow_G^* y$$

znamena, že z x lze dostat y pomocí nula nebo více přímých odvození.

Jazyk generovaný gramatikou G je

$$L(G) = \{w \in T^* : S \Rightarrow_G^* w\}.$$

Příklad 6.4 (Všechna binární slova). Gramatika

$$S \rightarrow 0S \mid 1S \mid \lambda$$

generuje jazyk $\{0,1\}^*$. Při každém kroku přidá na konec nulu nebo jedničku a pravidlem $S \rightarrow \lambda$ skončí. Například

$$S \Rightarrow 1S \Rightarrow 10S \Rightarrow 101S \Rightarrow 101.$$

Příklad 6.5 (Stejný počet blokových znaků). Gramatika

$$S \rightarrow 0S1 \mid \lambda$$

generuje jazyk

$$\{0^n 1^n : n \geq 0\}.$$

Každé použití prvního pravidla přidá jednu nulu vlevo a jednu jedničku vpravo. Třeba

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111.$$

Tento jazyk není regulární, takže gramatiky dovedou popisovat i jazyky, na které konečné automaty nestačí.

Příklad 6.6 (Dva nezávislé bloky). Gramatika

$$S \rightarrow aS \mid B, \quad B \rightarrow bB \mid \lambda$$

generuje $\{a^n b^m : n, m \geq 0\}$. Po přechodu z S na B už nelze přidat žádné další a , ale počty obou druhů znaků na sobě nezávisí.

Příklad 6.7 (Správně uzávorkované výrazy). Gramatika

$$S \rightarrow SS \mid (S) \mid \lambda$$

generuje správně uzávorkovaná slova. Pravidlo $S \rightarrow (S)$ vloží již hotový výraz do závorek a $S \rightarrow SS$ spojí dva výrazy za sebe. Například

$$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()(S) \Rightarrow ()((S)) \Rightarrow ()(()).$$

6.1 Derivační stromy

U gramatik, jejichž pravidla mají na levé straně jediný neterminál, můžeme průběh odvození znázornit stromem. Takový tvar mají všechna pravidla v našich dosavadních příkladech. Kořenem je počáteční neterminál. Použijeme-li pravidlo $A \rightarrow x_1 \cdots x_k$, přidáme vrcholu A zleva doprava syny $x_1, \dots, x_k$. Terminály v listech přečtené zleva doprava vytvoří výsledné slovo.

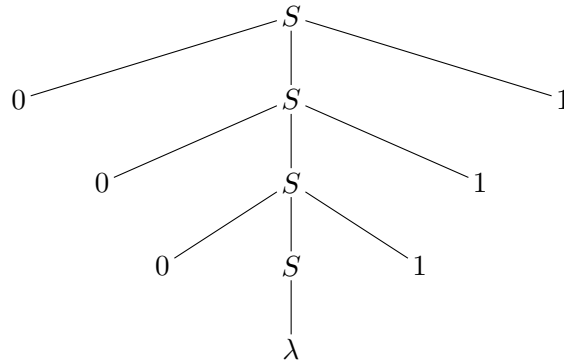


Figure 17: Derivační strom pro slovo 000111 v gramatice $S \rightarrow 0S1 \mid \lambda$.

Pořadí přepisování neterminálů se může lišit, aniž by se změnil derivační strom. U některých gramatik však může mít totéž slovo dva skutečně různé derivační stromy. Takovou gramatiku nazýváme *nejednoznačnou*. Třeba gramatika $S \rightarrow SS \mid a$ odvodí slovo aaa seskupením $(aa)a$ i $a(aa)$.

Příklad z přirozeného jazyka. Gramatika může popisovat i jednoduché věty:

$$\begin{aligned}
 \text{Věta} &\rightarrow \text{JmennáČást SlovesnáČást}, \\
 \text{JmennáČást} &\rightarrow \text{Člen PodstatnéJméno}, \\
 \text{SlovesnáČást} &\rightarrow \text{Sloveso JmennáČást}, \\
 \text{Člen} &\rightarrow \text{ten} \mid \text{jeden}, \\
 \text{PodstatnéJméno} &\rightarrow \text{algoritmus} \mid \text{automat}, \\
 \text{Sloveso} &\rightarrow \text{zkoumá} \mid \text{napodobuje}.
 \end{aligned}$$

Vygeneruje například větu „ten algoritmus zkoumá jeden automat“. Jde samozřejmě o hodně zjednodušený model češtiny; ukazuje ale, že pravidla určují nejen použitá slova, nýbrž i jejich stavbu.

6.2 Regulární gramatiky

Definice 6.8 (Regulární gramatika). Gramatika $G = (V, T, P, S)$ je *regulární*, pokud má každé její pravidlo jeden z tvarů

$$A \rightarrow wB \quad \text{nebo} \quad A \rightarrow w,$$

kde $A, B \in V$ a $w \in T^*$.

Na pravé straně je tedy nejvýše jeden neterminál a ten navíc stojí až na konci. Gramatika pro $0^n 1^n$ regulární není: pravidlo $S \rightarrow 0S1$ má terminál také za neterminálem. Naproti tomu gramatika pro $a^n b^m$ z předchozího příkladu regulární je.

Věta 6.9. Jazyk je regulární právě tehdy, když jej generuje nějaká regulární gramatika.

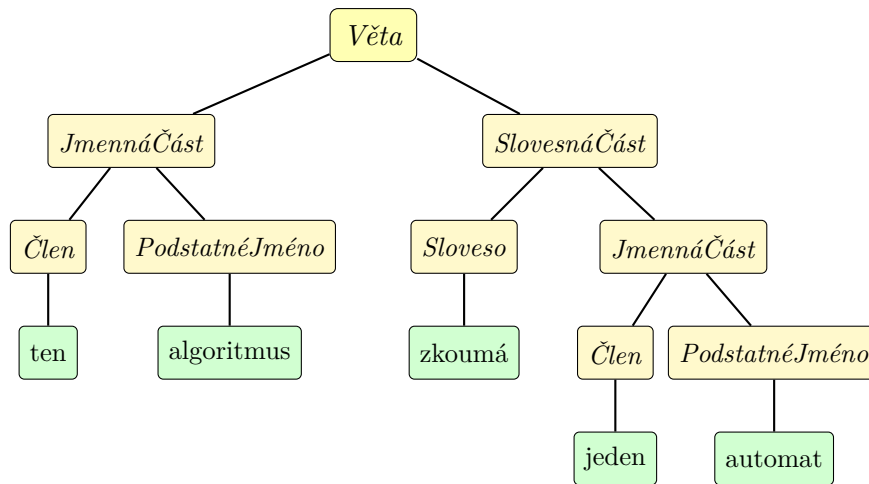


Figure 18: Derivační strom věty „ten algoritmus zkoumá jeden automat“.

Proof. Opět ukážeme konstrukci v obou směrech.

Z DFA na gramatiku. Mějme DFA $A = (Q, \Sigma, \delta, q_0, F)$. Pro každý stav $q \in Q$ vytvoříme stejnojmenný neterminál, terminály budou znaky z Σ a počátečním neterminálem bude q_0 . Za každý přechod $\delta(q, a) = r$ přidáme pravidlo

$$q \rightarrow ar$$

a za každý přijímající stav $f \in F$ pravidlo $f \rightarrow \lambda$. Všechna pravidla jsou regulární.

Každý krok automatu po znaku a odpovídá použití jednoho pravidla $q \rightarrow ar$. Slovo lze dovést z q_0 do přijímajícího stavu právě tehdy, když jej gramatika může vypsát a zbývající přijímající neterminál nakonec odstranit pravidlem $f \rightarrow \lambda$. Jazyky automatu a gramatiky jsou proto stejné.

Z gramatiky na automat. Mějme regulární gramatiku $G = (V, T, P, S)$. Neterminály použijeme jako stavy λ -NFA, stav S bude počáteční a přidáme jediný nový přijímající stav f .

Pro pravidlo

$$A \rightarrow a_1 a_2 \cdots a_k B$$

vytvoříme cestu ze stavu A do stavu B označenou postupně $a_1, \dots, a_k$. Je-li $k > 1$, vložíme na cestu nové pomocné stavy, které nepoužijeme u žádného jiného pravidla. Pro $k = 0$, tedy pro pravidlo $A \rightarrow B$, přidáme λ -přechod. Stejně zpracujeme pravidlo $A \rightarrow a_1 \cdots a_k$, jen cesta skončí v f ; pravidlo $A \rightarrow \lambda$ tedy vytvoří λ -přechod z A do f .

Použití pravidla v odvození přesně odpovídá průchodu po jemu přidělené cestě. Gramatika proto odvodí terminální slovo w právě tehdy, když sestrojený λ -NFA po přečtení w dojde do f . λ -NFA umíme převést na DFA, takže $L(G)$ je regulární. $\square$

Jeden jazyk třemi způsoby. Jazyk všech slov nad $\{a, b\}$ končících na ab můžeme popsat regulárním výrazem

$$(a \mid b)^* ab.$$

Přijímá jej také automat na obrázku 20.

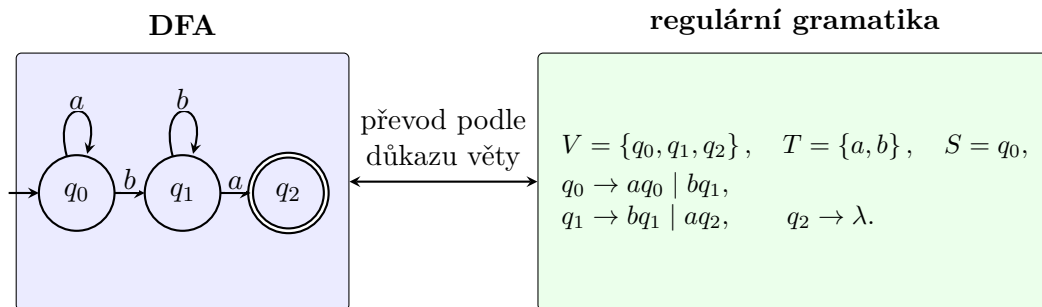


Figure 19: Ukázka převodu DFA na ekvivalentní regulární gramatiku.

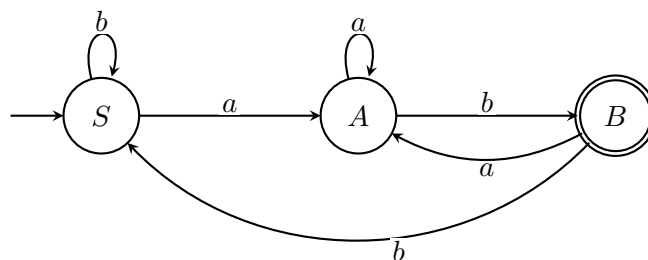


Figure 20: DFA pro slova končící na ab .

Stejný jazyk generuje regulární gramatika

$$\begin{aligned} S &\rightarrow aA \mid bS, \\ A &\rightarrow aA \mid bB, \\ B &\rightarrow aA \mid bS \mid \lambda. \end{aligned}$$

Neterminály S, A, B odpovídají stavům automatu. Každé pravidlo se znakem opisuje jeden přechod a pravidlo $B \rightarrow \lambda$ označuje přijímající stav.

Automat se hodí, když chceme slova rychle rozpoznávat. Regulární výraz bývá nejkratším popisem a gramatika je přirozená při generování. Kleeneho věta a věta 6.9 říkají, že u regulárních jazyků si můžeme vybrat ten pohled, který se zrovna hodí nejvíc.